Week #01

Date: 01

Software Design Concepts And Design Principles

1. Software Design :-

Software design is the process of transforming software requirements into a structured plan or blueprint that describes how the software will be built. It defines the software's:

- overall architecture,
- data and data structures,
- interfaces,
- components/modules,
- and relationship between different part of the system.

Software design begins after requirements have been analyzed and acts as the bridge between requirements and implementation

PAPER PRODUCT

Scanned with CamScanner

Date: 02

2. Why Software Design is Important :-

Software design allows engineers to think about the structure and quality of a system before committing everything to code. A good design :

- reduces complexity;
- makes software easy to understand;
- improves maintainability;
- makes testing and debugging easier;
- supports reuse;
- reduce the effect of future changes;
- provides a common understanding for developers;
- helps detect design problem before implementation;

3. Software Design Process :-

Software design is iterative, not a one-time activity. It starts with a high-level view of the entire system and gradually moves toward more detailed representations.

* The design should remain traceable to the original requirements throughout this process.

PAPER PRODUCT

Scanned with CamScanner

Date: 03

4. Elements of the Software Design Model :-

Pressman's design model contains four major design areas:

i Data/Class Design :-

Defines the data structure and design classes required by the software.

ii Architectural Design :-

Define the major components/subsystems of the system and the relationships between them.

iii Interface Design :-

Defines how :

- users interact with software,
- software communicates with external systems,
- internal components communicate.

iv Component-Level Design :-

Defines the internal details of individual components, including operations, algorithms and local data structures.

PAPER PRODUCT

Scanned with CamScanner

Date: 04

5. Characteristics of a Good Software Design :-

A basic good design should satisfy three basic goals:

1. It should correctly implement the explicit requirements and accommodate important implicit stakeholder requirements.
2. It should be understandable and useful to programmers, testers and maintainers.
3. It should give a complete implementation-oriented view of the systems data, functionality and behavior.

A good design should be : Correct + Complete + Understandable + Maintainable + Modular

6. Fundamental Software Design Concepts :-

These concepts form the foundation of good software design.

6.1 Abstraction :-

It means concentrating on the important features of something while hiding unnecessary details.

PAPER PRODUCT

Scanned with CamScanner

Date: 05

Design normally moves through different levels of abstraction:

High Abstraction "What does it do?"

More Detail

Low Abstraction "Exactly how will it be implemented?"

a) Procedural Abstraction :-

A named operation represents a sequence of detailed steps without exposing those steps.

Example : loginUser()

The name tells us the purpose without showing authentication queries, validation, and session creation.

b) Data Abstraction :-

A named collection of data represents a meaningful object.

Example : Student

The user of Student does not need to know every internal storage detail.
Abstraction helps engineers manage complexity by working with meaningful high-level concepts.

SP PAPER PRODUCT

Scanned with CamScanner

Date: 06

7.2 Architecture :-

Software architecture is the overall structure and organization of a software system, including its major components and how those components interact.

Example of simple university system :

Architecture provides the - picture structure on which more detailed software design is built.

7.3 Design Patterns :-

A design pattern is a proven general solution to a recuring design problem in a particular context.

A pattern is not ready - made code. It provides a reusable design idea or structure that can be adopted to different systems.

PAPER PRODUCT

Scanned with CamScanner

Date: 07

7.4 Separation of Corcents :- (SoC)

means dividing a complex problem into smaller parts so that each concern can be handled as independently as possible.
A concern is a particular feature, responsibility or behavior of the software.

Example: Instead of one huge university portal handling everything together:

University Portal

login Courses Fees Result

Each concern can be understood and developed more independtly. This follows the basic idea of divide and conquer.

7.5 Modularity :-

Modularity means dividing software into separately named and manageable components called modules, which work together to satisfy the system requirements.

Example :- E-Commerce System

- Order Module
- Payment Module
- Cart Module

PAPER PRODUCT

Scanned with CamScanner

Date: 08

Benifits of Modularity :-

It makes:

- development easier to plan;
- changes easier to make;
- testing and debugging easier;
- modules easier to understand;
- maintenance safer;
- software increments easier to develop.

However, more modules are not always better. Too few modules produce highly complex components, while too many modules increase integration complexity and cost.

Modularity and Software Cost :-

- It shows that module cost tends to fall as modules increase, while integration cost rises. The desired region is around M, where total development cost is lowest.

* Avoid both undermodularity and overmodularity.

SP PAPER PRODUCT

Scanned with CamScanner

Date: 09

7.6 Information Hiding :-

It means hiding the data and processing details of a module from other modules that do not need that information. A module should expose only what the other modules need through a controlled interface.

Example :- Bank Account

Other modules use the public operations without depending on the internal implementation.

Information hiding reduces the chance that changes or errors inside one module will effect many other part of the system.

Hidden:
balance
transaction logic
validation rules
Public Interface:
deposit()
withdraw()
getBalance()

7.7 Functional Independence :-

Functional independence means designing modules so that each module performs a focused function while having minimal unnecessary interaction with other modules. It is achieved mainly through :

i) High Cohesion :-

Cohesion measures how strongly the elements inside a module relate to one purpose.

PAPER PRODUCT

Scanned with CamScanner

Example: PaymentService handles payment-related work.

ii) Low Coupling :-

Coupling measures how strongly one module depends on other modules.

* Good software design therefore aims for: High Cohesion + Low Coupling.

7.8 Refinement :-

Refinement is the process of gradually developing a high-level design into increasingly detailed design.
It is commonly stepwise refinement.

Refinement is a top-down strategy: begin with the overall function and repeatedly add detail until implementation-level operations are reached.

7.9 Aspects and Crosscutting Concerns :-

Some concerns affect many different parts of a system instead of belonging to only one module. These are called crosscutting concerns.

Examples include:

PAPER PRODUCT

Scanned with CamScanner

Date: 11

- security
- authentication
- logging

Suppose authentication is required before many different functions:

Authentication

Courses Fees Results

An aspect represents such as crosscutting concern so that it can be handled systematically rather than duplicated throughout many components.

7.10 Refactoring :-

Refactoring means improving the internal structure of an existing design or code without changing its externally visible behavior or functionality.

Example :-

- reducing unnecessary dependencies;
- removing duplicated logic;
- splitting an oversized module;
- simplifying complicated structures;
- improving cohesion.

The functionality remains the same, but the internal design becomes cleaner and easier to maintain.

PAPER PRODUCT

Scanned with CamScanner

8. Design Quality - FURPS :-

Pressman presents FURPS as a set of software quality attributes that should be considered during design.

Functionality : Features, capabilities & security.

Usuability : Human factors, consistency, asthetics documentation.

Reliability : Correct and dependable operation, failure and recovery characteristics.

Performance : Speed, response time, throughput and resource usage.

Supportability : Maintainability, testability, adaptability, compatibility, etc.

The important principle is that quality should be considered while designing, not added after implementation.

9. Design Principles/Quality Guidelines :-

A good software design should :

SP PAPER PRODUCT

Scanned with CamScanner

Date: 13

1. Use a recognizable and suitable architecture, architectural style or pattern.
2. Be modular, dividing the system logically into manageable element or subsystems.
3. Provide distinct representation of data, architecture, interfaces and components.
4. Use appropriate data structures for the classes and information being implemented.
5. Create components with independent functional characteristics.
6. Design interfaces that minimize unnecessary complexity between components & the external environment.
7. Be created using a repeatable method based on requirements - analysis information.
8. Be represented using notation that communicates the design clearly and effectively.

PAPER PRODUCT

Scanned with CamScanner

Week #02

Date: 14

Object-Oriented Design With UML And System Design :-

1. Object-Oriented Design (OOD) :-

It is a software design approach in which a system is designed as a collection of interacting objects. Each object:

- stores its own data/state;
- provides operations/methods on that data;
- interacts with other objects to perform system functions.

Classes define the structure and behavior of objects, and objects are created from these class definitions during program execution.

Example :-

For a University Portal, possible objects/classes are:

- Student
- Course
- Instructor
- Enrollement
- Payment

Each object has both data and operation.

PAPER PRODUCT

Scanned with CamScanner

Date: 15

2. Unified Modeling Language (UML):-

It is a standard graphical modeling language used to visualize, specify and document software systems, especially object-oriented systems.

UML is not a programming language. It provides different types of diagrams for representing different views of a system.

UML contains 13 diagram types, although a smaller group is commonly needed for most software models.

Why UML is Used?

UML helps developers:

- understand a system before coding;
- visualize classes, objects and relationships;
- model system behavior and interactions;
- communicate design decisions;
- document the software design.

Important UML Diagrams :-

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- State Diagram

SP PAPER PRODUCT

Scanned with CamScanner

Date: 16

3. Structural & Dynamic Models:-

UML design models can broadly be divided into two types.

a) Structural Models:-

Show the static organization of a system. They represent things such as:

- classes
- objects
- inheritance and composition
- components
- relationships

Example: Class Diagram

b) Dynamic Models:-

Shows what happens during system execution. They represents:

- interactions between objects;
- sequence of messages;
- changes in object states.

Example: Sequence Diagram and State Machine Diagram.

PAPER PRODUCT

Scanned with CamScanner

Date: 17

4. Object-Oriented Design Process :-

i) Understand System Context :-

Identify :

- the system boundary;
- external users or systems;
- how the system interacts with its environment.

ii) Design System Architecture :-

Decide the major parts or subsystems and how they will work together.

iii) Identify Principal Objects :-

Identify the important object/classes required by the system.

iv) Develop Design Models :-

Use UML models to represent the structure and behavior of the system.

v) Specify Interfaces :-

Define how objects, components and external systems communicate with each other.

PAPER PRODUCT

Scanned with CamScanner

Date: 18

5. System Design :-

System Design is the process of deciding how the complete software system will be organized to satisfy its requirements.

It moves from the requirements towards a technical solution by deciding :

- major subsystems/components;
- their responsibilities;
- relationships and communication;
- overall organization of the system.

At this stage, we are concerned more with the big picture than the individual algorithms or line of code.

6. Software Architecture :-

Software architecture represents the high-level organization of a software system, its major components and the relationship between them.

Architecture is an important part of system design because system design must decide the overall structure before detailed components and classes are designed.

PAPER PRODUCT

Scanned with CamScanner

Date: 19

System design decides the overall solution, and Software Architecture represents the high-level structure of that solution.

7. Relationship Between OOD and System Design:-

System Design:-

It decides the overall organization of the system.

Objected-Oriented Design:-

It decides the classes/objects and their relationships inside that system.

A simple flow is:

Requirements → System Design → Object-Oriented Design → Detailed Object Design → Code

SP PAPER PRODUCT

Scanned with CamScanner

Week #03

Date: 20

Object Design & Mapping Design to Code :-

1. Object Design :-

Object design is the process of refining the objects/classes identified during analysis into detailed design classes that are ready for implementation.

At the analysis level, classes mainly describe things from the problem domain. During design, more detail is added, such as:

- attributes and their data types,
- operations / methods,
- relationships with other classes,
- interfaces,
- data structures,
- and implementation-related responsibilities.

2. Identifying Object Classes :-

Before objects can be designed, the important classes in the system must be identified.

i) Grammatical Analysis :-

Analyze the requirements description :

PAPER PRODUCT

Scanned with CamScanner

- Nouns may indicate object/classes or attributes.
- Verbs may indicate operations/services.

Example: "A student registers for a course."

Possible classes: Student, Course
Possible operation: registerCourse()

ii) Tangible and Domain Entities :-

Look for important things in the application domain such as:

- people or roles
- physical objects
- organizational units
- locations
- events

iii) Scenario-Based Identification :-

Study use cases or system scenarios and identify the: objects + attributes + operations needed to perform each scenario. In practice, these approaches are normally combined rather than relying on only one method.

3. Designing an Object Class :-

After identifying a class, its designe must be refined. For each important class determine:

PAPER PRODUCT

Scanned with CamScanner

Date: 22

Class Name – What concept does the class represent?

Responsibilities – What is the class responsible for doing?

Attributes – What information/state must it store?

Operations/Methods – What behavior should the class provide?

Relationships – Which other classes must it collaborate with?

Interfaces – Which operations should be available to other objects/components?

A detailed UML class represents its name, attributes and operations in separate sections. Operations in an OO programming language later become methods.

4. Types of Design Classes :-

- User Interface Classes : Handle interaction between users and the system.
- Business Domain Classes : Represent important business/domain objects.

PAPER PRODUCT

Scanned with CamScanner

- Process Classes: Handle lower-level business processing.
- Persistent Classes: Represent data that must remain stored, such as database data.
- System Classes: Provide system management, control and communication functions.

5. Characteristics of a Well-Formed Design Class:-

A good design class should have four important characteristics.

Complete and Sufficient:-

The class should contain everything reasonably necessary to fullfil its purpose, but nothing unnecessary.

Primitiveness:-

Each method should perform one clear service. There should not be several unnecessary ways of performing the same basic operation.

High Cohesion:-

The class should have a small and focused set of related responsibilities.

SP PAPER PRODUCT

Scanned with CamScanner

Date: 24

Low Coupling :-

The class should have only the necessary dependencies on other classes.

6. Interface Specification :-

An interface defines the externally visible operations/services through which other objects or components can use a class.

An interface specifies what services are available without revealing the internal implementation.

- it contains publicly available operations;
- internal data representation should remain hidden;
- in UML, an interface does not contain an attribute section;
- an object can provide more than 1 interface;
- several objects may also be accessed through the same interface.

7. Mapping Design to Code :-

Mapping design to code means transforming the detailed software design into implementation using an object-oriented programming language.

At this stage, the design moves from

PAPER PRODUCT

Scanned with CamScanner

Date: 25

abstractions such as classes, attributes, operations, and interfaces toward their programming language equivalents.

UML/Design → Code Mapping :-

Design Element	OO Code
Class	Class definition
Object	instance of a class
Attribute	Field / instance variable
Operation	Method
Interface	Interface definition
Association	Object reference
Generalization	Inheritance
Visibility	Access modifier
Message between Objects	Method call
1 to many relationship	Usually reference to a collection

8. Example: Design Class to Java Code :-

Suppose the UML design Contains :

Student :	
Attributes :	A simple Java implementation may be.
• studentId : int	
• name : String	
Operations :	
• registerCourse()	
• viewResult()	

PAPER PRODUCT

Scanned with CamScanner

```
class Student {
    private int studentId;
    private String name;

    public void registerCourse() {
        // registration logic
    }
    public void viewResult() {
        // result Logic
    }
}
```

The mapping is straightforward:

- Student → Java class
- studentId, name → fields
- registerCourse(), viewResult() → methods
- private/public UML visibility → access modifiers.

The exact implementation may differ between programming languages, but the design provides the structure that guides implementation.

9: Component-Level Design & Code:-

As the design approaches implementation; component-level design provides enough detail for coding and testing. It defines:

- internal data structures,
- algorithms,

PAPER PRODUCT

Scanned with CamScanner

Date: 27

- interfaces
- communication mechanisms,
- and processing logic for individual components.

The architecture tells us where a component belongs, while detailed object/component design tells us how that component will actually work.

10. Design and Code Are Not Always 1:1 :-

A design model guides implementation, but it does not mean that every diagram element must always become exactly one program element.

For Example:

- one object may provide several interfaces;
- one interface may be implemented by different classes;
- additional implementation classes may be required for validation, searching or infrastructure services;
- implementation decisions may add detail that was unnecessary at the higher design level.

Therefore: "Design guides code; it does not mechanically replace coding decisions."

PAPER PRODUCT

Scanned with CamScanner

Week #04

Date: 28

1. User Interface Design :-

User Interface (UI) is the part of a software system through which a user interacts with the application.

User Interface Design is concerned with designing this interaction so that users can access system functions and information in a clear and usable way.

Examples of UI elements include :

- screen and pages,
- menus,
- buttons,
- forms,
- input fields,
- navigation controls,
- messages and displayed information.

2. Main Elements of UI Design :-

UI design involves 3 broad kinds of elements.

i) Aesthetic Elements :-

Concern the visual appearance of the interface, such as :

- layout
- colors
- graphics
- visual representation

PAPER PRODUCT

Scanned with CamScanner

Date: 29

ii) Ergonomic Elements :-

Concern how information and controls are arranged for convenient human use, including:

- information placement,
- navigation,
- interaction mechanism,
- suitable interface metaphors

iii) Technical Elements :-

Concern the software mechanisms used to construct the UI, such as:

- UI patterns,
- reusable interface components.

* A successful interface therefore should not only look good; it must also support effective interaction and be technically well designed.

3. User Interface Classes :-

In object-oriented design, user interface classes represent the software abstractions needed for Human-Computer Interaction (HCI).

They may represent things such as:

- screens
- forms
- menus
- controls

SP PAPER PRODUCT

Scanned with CamScanner

Date: 30

- other visual interaction elements.

Example:-

For a University Portal, possible VI classes may correspond to:

- Login Screen
- Course Registration Screen
- Result Screen
- Payment Screen

The VI presents information and collects user actions while the underlying business classes perform the actual application processing.

4. VI as a Subsystem:-

A user interface can itself be considered a separate subsystem within the overall application architecture.

This is useful because the responsibilities can remain separated:

VI → handles interaction and presentation.

Application Logic → performs business operations

Data Layer → manages stored information

PAPER PRODUCT

Scanned with CamScanner

This separation prevents presentation details from becoming mixed unnecessarily with the rest of the system logic.

User Interface

Business Logic

Data Layer

5. Basic UI Design Considerations

The important UI considerations can be grouped as :

- **Presentation :-** How information, graphics and controls are visually arranged.
- **Interaction :-** How the user provides input and performs actions.
- **Navigation :-** How users move between different parts of the application.
- **Information Layout :-** Where important information and controls are positioned.

SP PAPER PRODUCT

Scanned with CamScanner

- **Reuse :-**
  Using established UI patterns and reusable components where appropriate.

6. Persistent Layer Design :-

Most useful applications need some information to remain available after the program stops executing. This is called persistent data.

Examples :-

- student records,
- registered courses,
- customer accounts,
- orders,
- payment records,

A persistent layer is the part of the software design responsible for managing such long-term stored data.

Pressman defines persistent classes representing data stores, such as databases, whose information persists beyond one execution of the software.

SP PAPER PRODUCT

Scanned with CamScanner

Date: 33

7. Persistent Data Sources :-

A persistent data source stores information so that it remains available even after the program stops executing..

Common persistent data sources include :

- Databases – store structured application data permanently.
- Files – store information directly in the file system.

8. Purpose of the Persistent Layer :-

The persistent layer separates the application's business logic from the details of permanent data storage.

It's main responsibilities are :

- storing data,
- retrieving data,
- updating data,
- deleting data
- providing controlled access to stored information.

PAPER PRODUCT

Scanned with CamScanner

9. Persistant Classes :-

A persistent class represents or manages information that must continue to exist after the software finishes executing.

For example, a student object may temporarily exist in memory while the application is running, but the student's record must remain stored permanently.

Persistent classes are therefore associated with long-term data storage such as databases.

Example :-

Possible persistent classes in a university system:

- Student
- Course
- Enrollment
- Result

10. Data Management Classes :-

Some classes are created specifically to handle communication between the application and persistent storage.

These may perform operations such as:

- saveStudent()

PAPER PRODUCT

Scanned with CamScanner

- findStudent()
- updateStudent()
- deleteStudent()

Such classes keep storage-related responsibilites separate from user interface and business-processing classes.

Data-management components are part of the infrastructure required during detailed software design.

11. Data Design at Different Levels :-

Data design can occur at different levels.

a) Application/Architectural Level :-

The focus is on persistent storage such as:

- databases;
- files;
- overall organization of application data.

b) Component Level :-

This focus is on:

- local data structures;
- data types,
- internal data used by individual components,
- algorithms that manipulate those structures.

SP PAPER PRODUCT

Scanned with CamScanner